AppSec / ProdSec / Security Engineer

Comprehensive Interview Preparation Guide

100 + Technical Q&A | Behavioral & HR Questions | Bonus Topics

Covers: OWASP Top 10 • Cryptography • Network Security • Web Vulnerabilities • Mobile Security • Cloud Security • SAST/DAST/SCA • Authentication • Secure SDLC • Tools & Automation • Behavioral / Situational Questions

Your questions (from notes) + Curated additions by Claude

Table of Contents

PART A TECHNICAL QUESTIONS

1. Cryptography & PKI
2. Web Application Security (OWASP, Injections, XSS, CSRF…)
3. Authentication & Session Management
4. Network & Protocol Security
5. Secure SDLC, SAST/DAST/SCA
6. Mobile Security (iOS / Android)
7. Cloud & Container Security
8. Tools & Automation
9. Miscellaneous Technical Topics

PART B BEHAVIORAL / HR / SITUATIONAL QUESTIONS

PART C QUESTIONS TO ASK YOUR INTERVIEWER

PART D BONUS QUESTIONS (Claude's Additions)

PART A — TECHNICAL QUESTIONS

1. Cryptography & PKI

Q1. What is Cryptography in Cybersecurity?

Cryptography is the practice of securing information by transforming it into an unreadable format using mathematical algorithms. In cybersecurity it provides Confidentiality (encryption), Integrity (hashing/MACs), Authentication (digital signatures), and Non-repudiation. Key types: symmetric (AES, DES), asymmetric (RSA, ECC), and hashing (SHA-256, MD5). TLS, SSH, and JWT all rely on cryptographic primitives.

Q2. Hash vs Encryption vs Encoding – what is the difference?

Hashing: one-way deterministic function (SHA-256, bcrypt). Cannot be reversed. Used for password storage and integrity checks. Encryption: two-way, requires a key (AES-256, RSA). Reversible with the correct key. Used for data confidentiality. Encoding: not a security control—it is simply a format transformation (Base64, URL-encoding) meant for data transport, fully reversible without any key. Interview tip: encoding != encryption; never store passwords using Base64.

Q3. Explain SSL/TLS and how it works.

TLS (successor to SSL) establishes an encrypted channel over TCP. Handshake: (1) Client Hello – supported cipher suites + random nonce. (2) Server Hello – chosen cipher + certificate. (3) Client validates certificate against trusted CA. (4) Key exchange (ECDHE for forward secrecy). (5) Both sides derive session keys. (6) Finished messages (MAC verified). Data is then encrypted with symmetric key (AES-GCM). Common attacks: POODLE (SSLv3), BEAST (TLS 1.0), CRIME (compression), downgrade attacks. Best practice: enforce TLS 1.2+ and disable weak ciphers.

Q4. What is the 3-Way TCP Handshake?

SYN → SYN-ACK → ACK. Security relevance: SYN flood attacks exploit the half-open state before ACK. Mitigation: SYN cookies, rate-limiting, firewall state tables. In TLS, the TCP handshake precedes the TLS handshake.

Q5. What is SSL/TLS in mobile apps?

Mobile apps must implement certificate pinning to prevent MITM even when a rogue CA is trusted by the device. Android: use network_security_config.xml or OkHttp CertificatePinner. iOS: use URLSession with didReceiveChallenge. Bypass techniques (Frida, SSL Kill Switch) must be considered in mobile pen-testing. Also ensure TLS 1.2+, disable cleartext traffic.

Q6. Explain SAML / OAuth 2.0 / OpenID Connect and their differences.

SAML: XML-based federation for SSO, common in enterprise (IdP → SP assertion). OAuth 2.0: authorization framework; delegates access via access tokens (not authentication). Flows: Authorization Code (safest), Implicit (deprecated), Client Credentials, Device Code. OpenID Connect (OIDC): authentication layer on top of OAuth 2.0; adds ID token (JWT) containing user identity. Common attacks: open redirect, token leakage, CSRF on redirect_uri, implicit flow token theft.

2. Web Application Security

Q7. What is the CIA Triad?

Confidentiality: only authorised parties can read data (encryption, ACLs). Integrity: data is not tampered with (hashing, digital signatures, checksums). Availability: systems are accessible when needed (redundancy, DDoS mitigation, backups). A security engineer maps every control to at least one pillar.

Q8. What is OWASP Top 10 (2021)?

A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection, A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable & Outdated Components, A07 Identification & Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging & Monitoring Failures, A10 Server-Side Request Forgery. Know each category's root cause, example, and remediation.

Q9. DOM-based XSS vs Reflected XSS vs Stored XSS – explain all three.

Stored XSS: payload persisted in DB, served to all users—highest severity. Reflected XSS: payload in request, echoed in response; requires victim to click a crafted link. DOM-based XSS: payload never sent to server; JavaScript reads attacker-controlled source (location.hash, document.referrer) and writes to a dangerous sink (innerHTML, eval). Prevention: output encoding, CSP, avoid dangerous sinks, use textContent instead of innerHTML.

Q10. What is CSRF and why does it happen?

Cross-Site Request Forgery tricks an authenticated user's browser into sending an unwanted state-changing request to a target site. It happens because browsers automatically attach cookies to cross-origin requests. The site doesn't distinguish the forged request from a legitimate one. Mitigations: synchronizer token pattern, Double-Submit Cookie, SameSite=Strict/Lax cookies, checking Origin/Referer headers.

Q11. CSRF bypass techniques when a CSRF token is present.

(1) Token not tied to session – reuse another user's token. (2) Token in URL – leaked via Referer header. (3) CSRF token in JSON body but CORS misconfiguration allows cross-origin reads. (4) Clickjacking to trigger form submission. (5) XSS to extract and replay token. (6) Change Content-Type to text/plain if server doesn't strictly validate.

Q12. SSRF – explain exploitation.

Server-Side Request Forgery: attacker makes the server issue requests to internal resources. Impact: access AWS metadata (169.254.169.254), internal services, RCE via Redis/Memcached. Blind SSRF: no response returned—detect via out-of-band (Burp Collaborator/interactsh). Bypass filters: IP obfuscation (decimal, hex, IPv6, 0.0.0.0), DNS rebinding, open redirects, protocol smuggling (gopher://). Mitigations: allowlist, disable redirects, network segmentation.

Q13. SSRF Mitigation.

Use allowlists over blocklists. Disable unnecessary URL schemes (gopher, file, dict). Validate and resolve DNS before allowlist check (prevent DNS rebinding—resolve twice). Deploy WAF rules. Segment server network so backend services are unreachable from the app tier. Use metadata endpoint protection on cloud (IMDSv2 on AWS requires PUT + session token).

Q14. SQL Injection – types and prevention.

Classic (error-based), Blind (boolean/time-based), Out-of-Band, Second-Order (stored then executed), Union-based. Prevention: parameterised queries / prepared statements (primary), stored procedures with bind variables, input validation, least privilege DB user, WAF. Second-order: validate on retrieval, not just insertion.

Q15. HTTP Parameter Pollution (HPP).

Injecting duplicate parameters to confuse server-side parsing. Example: ?role=user&role=admin. Behaviour varies by framework (PHP takes last, ASP.NET concatenates). Can bypass WAF signatures, override parameters in signed requests. Test: send duplicate params with different values.

Q16. Double-Cookie pattern.

CSRF mitigation: set a random token in a cookie AND require the same value in a request parameter/header. The server validates they match. Since an attacker cannot read the cookie (same-origin policy), they cannot reproduce the token. Secure variant requires HTTPS + Secure cookie flag.

Q17. CORS vs SOP – differences.

Same-Origin Policy (SOP): browser enforces that scripts on origin A cannot read responses from origin B. CORS: server-side HTTP headers that selectively relax SOP. Misconfiguration: Access-Control-Allow-Origin: * with Access-Control-Allow-Credentials: true is invalid but some servers reflect the Origin header—leads to cross-origin credential theft. Test: send Origin: evil.com and check if it's reflected with Allow-Credentials: true.

Q18. CSP – Content Security Policy.

HTTP response header that whitelists sources for scripts, styles, images etc. Prevents XSS by blocking inline scripts and untrusted domains. Directives: default-src, script-src, style-src, connect-src, frame-ancestors. Pitfalls: unsafe-inline, unsafe-eval, wildcard (*) in script-src, whitelisting CDN domains that host JSONP endpoints (bypassable). Report-only mode for testing.

Q19. CRLF Injection.

Attacker injects \r\n into HTTP headers to split the response, inject arbitrary headers, or perform HTTP response splitting leading to XSS or cache poisoning. Example: Location: /page\r\nSet-Cookie: session=evil. Mitigation: strip/encode CR+LF from user input used in HTTP headers.

Q20. SSRF vs CORS vs SOP in the context of same-origin.

SOP prevents browser scripts from reading cross-origin responses. CORS is the mechanism to selectively allow it. SSRF is server-side – the server itself makes requests to other servers, bypassing SOP entirely because SOP only applies to browsers.

Q21. Explain HTTP Request Smuggling (De-sync Attack).

Occurs when front-end (load balancer/CDN) and back-end servers disagree on where one HTTP request ends and the next begins due to ambiguous Content-Length vs Transfer-Encoding: chunked headers. TE.CL, CL.TE, TE.TE variants. Impact: bypass security controls, poison other users' requests, cache poisoning, credential hijacking. Detection: use Burp Suite HTTP Request Smuggler extension.

Q22. What is Prototype Pollution?

JavaScript vulnerability: attacker modifies Object.prototype so all objects inherit malicious properties. Via JSON merge (e.g., __proto__.admin=true) or recursive merge in lodash. Impact: privilege escalation, RCE in Node.js (child_process gadgets). Fix: use Object.create(null), validate keys, use Map/Set instead of plain objects, freeze Object.prototype.

Q23. XML Injection / XXE.

XXE (XML External Entity): attacker defines external entity in XML that the parser resolves. Can read local files (/etc/passwd), SSRF, DoS (Billion Laughs). Mitigation: disable external entity processing (set FEATURE_SECURE_PROCESSING), use JSON, whitelist input. XML Injection: injecting XML syntax to break structure or add elements.

Q24. Explain Race Condition vulnerability.

TOCTOU (Time of Check to Time of Use): gap between checking a condition and using a resource. Example: two concurrent withdrawal requests both pass balance check before either deducts. Exploitation: send many simultaneous requests (Turbo Intruder, ffuf). Mitigation: database-level locking, atomic operations, idempotency tokens.

Q25. LFI vs RFI vs Directory Traversal.

LFI (Local File Inclusion): include files already on server via path manipulation (?file=../../etc/passwd). RFI (Remote File Inclusion): include files from attacker-controlled URL (requires allow_url_include=On in PHP—rare). Directory Traversal: read arbitrary files without code execution. Bypass: URL encoding (%2e%2e%2f), double encoding, null byte (%00 in older PHP). Mitigation: realpath() canonicalization, whitelist allowed files.

Q26. Remediation for Missing Security Headers and XSS.

Missing headers: add Content-Security-Policy, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Strict-Transport-Security (HSTS), Referrer-Policy, Permissions-Policy. XSS remediation: context-aware output encoding (HTML, JS, URL, CSS contexts), CSP, avoid innerHTML/document.write, sanitize with DOMPurify, HTTPOnly cookies.

Q27. What is SSTI (Server-Side Template Injection)?

Attacker injects template syntax ({{7*7}}) into user-controlled data that is rendered by a template engine (Jinja2, Twig, Freemarker). Can lead to RCE. Detection: {{7*7}} $\rightarrow$ 49. Escalate to RCE via engine-specific gadgets (config.__class__.__init__.__globals__['os'].popen). Fix: never pass user input directly to template render(); use sandboxed environments.

Q28. CORS misconfiguration and SQLi in CORS.

A CORS misconfiguration (reflect Origin + Allow-Credentials: true) lets an attacker read authenticated responses. SQLi in CORS: if the Origin header value is stored in DB or used in queries without sanitisation, it becomes an injection vector. Always validate Origin against a strict allowlist, never use it in SQL queries.

Q29. What is 'or 1=1' – how does it work?

Classic SQLi tautology. If a query is SELECT * FROM users WHERE username='INPUT', injecting ' or '1'='1 makes it always true, returning all rows. Modern parameterised queries prevent this. Also test: ' or 1=1-- , admin'--. Understanding the underlying query context is key for successful injection.

---

3. Authentication & Session Management

Q30. Session Hijacking – how does it work and how to prevent it?

Attacker steals a valid session ID via XSS (document.cookie), network sniffing (HTTP), predictable session IDs, or session fixation. Prevention: HTTPOnly + Secure cookie flags, regenerate session ID on privilege change, short session timeouts, bind session to IP/User-Agent (soft binding), use SameSite=Strict.

Q31. Cookie Attributes – explain each.

HttpOnly: prevents JS access (mitigates XSS cookie theft). Secure: only sent over HTTPS. SameSite=Strict: never sent cross-site. SameSite=Lax: sent on top-level navigation GET. SameSite=None: sent always (requires Secure). Domain: scope of cookie. Path: URL path scope. Expires/Max-Age: persistence. __Host- prefix: forces Secure + no Domain + Path=/.

Q32. Session Fixation.

Attacker sets a known session ID before login (via URL parameter or cookie injection), victim logs in with that ID, attacker reuses it. Fix: always regenerate session ID after successful authentication.

Q33. Token-based Authentication vs Session-based Authentication.

Session-based: server stores session state; client holds session ID cookie; stateful. Token-based (JWT): server is stateless; client stores signed token (usually in localStorage or cookie); verified via signature. JWT advantages: scalable, works across domains. Risks: alg:none attack, weak secret, storing in localStorage (XSS risk). Prefer httpOnly cookie for JWT storage.

Q34. JWT Token Leak – impact and mitigation.

If JWT is in localStorage, XSS can steal it → full account takeover. If in URL, logged in server logs. Mitigation: store in httpOnly cookie, short expiry, token binding, check jti claim for revocation (token blacklist or use short-lived access + refresh rotation).

Q35. CSRF prerequisites (JWT token context).

CSRF requires the victim's credentials to be automatically attached. Cookies are auto-sent → CSRF possible. JWT in Authorization header → NOT automatically sent by browser → not CSRF-vulnerable. But if JWT stored in cookie → CSRF applies. Best: use SameSite=Strict + CSRF token.

Q36. CyberArk Conjur – what is it?

CyberArk Conjur is a secrets management platform for DevOps. Stores credentials, API keys, SSH keys in a centralised vault. Applications retrieve secrets at runtime via REST API or SDK (no hardcoded credentials). Supports RBAC, audit logs, automatic rotation. Conjur is the open-source version; PAM is the full enterprise product.

---

4. Network & Protocol Security

Q37. Important Port Numbers for Security Professionals.

21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 80 HTTP, 110 POP3, 143 IMAP, 389 LDAP, 443 HTTPS, 445 SMB, 636 LDAPS, 1433 MSSQL, 1521 Oracle, 3306 MySQL, 3389 RDP, 5432 PostgreSQL, 5900 VNC, 6379 Redis, 8080/8443 Alt HTTP/HTTPS, 27017 MongoDB. Know TCP vs UDP per service.

Q38. DNS vs DHCP.

DNS: resolves domain names to IP addresses (UDP/TCP 53). Attacks: DNS poisoning, DNS rebinding, DNS tunnelling. DNSSEC for integrity. DHCP: dynamically assigns IP config to clients (UDP 67/68). Attacks: DHCP starvation, rogue DHCP server (MITM). DHCP snooping at switch level mitigates rogue servers.

Q39. REST API vs SOAP API – security differences.

REST: lightweight, JSON/XML, stateless, uses HTTP verbs. SOAP: XML-based, uses WSDL, stricter contract. Security: SOAP has WS-Security (built-in signing/encryption). REST relies on TLS + OAuth. REST attack surface: IDOR, broken auth, mass assignment, BOLA. SOAP: XXE in XML body, injection in SOAP parameters. Both: test for excessive data exposure, rate limiting, input validation.

Q40. What happens when you type a domain in the browser?

(1) Check browser DNS cache. (2) OS resolver / hosts file. (3) DNS recursive lookup → root → TLD → authoritative NS. (4) TCP handshake to resolved IP. (5) TLS handshake (HTTPS). (6) HTTP GET request. (7) Server processes, returns HTML. (8) Browser parses HTML, fetches sub-resources (CSS/JS/images), renders page. Security hooks: DNS over HTTPS, HSTS preload, certificate transparency, CORS headers on sub-resources.

Q41. Nmap commands for security testing.

nmap -sV -sC target (default scripts + version). nmap -p- target (all 65535 ports). nmap -sU -p 53,161 target (UDP). nmap -O target (OS detection). nmap --script vuln target (vuln scripts). nmap -sn 192.168.1.0/24 (host discovery/ping sweep). Always get written permission before scanning.

Q42. Persistent Cookie.

Cookie with an Expires or Max-Age attribute; stored on disk and survives browser restart. Session cookie has no expiry, deleted when browser closes. Security risk: persistent cookies extend attack window if stolen. Use session cookies for auth tokens or short Max-Age; always set Secure + HttpOnly.

Q43. SOP vs CORS (detailed).

SOP blocks cross-origin reads at browser level. CORS headers (sent by server) tell browser which origins may read responses. Simple requests (GET/POST with simple content types) are sent but response blocked unless CORS allows. Preflighted requests (OPTIONS) check permission before sending. Credential-bearing requests need explicit Allow-Credentials: true AND non-wildcard origin.

---

5. Secure SDLC, SAST / DAST / SCA

Q44. What is SSDLC (Secure Software Development Life Cycle)?

Integrating security at every SDLC phase: Requirements (threat modelling, abuse cases), Design (security architecture review), Development (secure coding standards, peer review, SAST in IDE), Testing (DAST, penetration test), Deployment (hardening, secrets management), Maintenance (patch management, vulnerability disclosure). Shift-left means moving security earlier (IDE plugins, pre-commit hooks).

Q45. SAST vs DAST – differences.

SAST (Static Application Security Testing): analyses source code/bytecode without running the app. Finds issues early, high false-positive rate, language-specific. Tools: SonarQube, Semgrep, Checkmarx, Veracode. DAST (Dynamic Application Security Testing): tests running application from outside like an attacker. Finds runtime issues (auth, injection, config). Tools: OWASP ZAP, Burp Suite, Nikto. IAST combines both (agent inside running app).

Q46. SCA (Software Composition Analysis).

Identifies open-source libraries and their known vulnerabilities (CVEs). Checks licence compliance. Tools: Snyk, OWASP Dependency-Check, Black Duck, GitHub Dependabot. Key: maintain SBOM (Software Bill of Materials). Integrates into CI/CD to block builds with critical CVEs.

Q47. Shift-Left – SAST/SCA in CI/CD.

Run SAST and SCA on every pull request. IDE plugins (SonarLint, Snyk IDE) for real-time feedback. Pre-commit hooks (gitleaks for secrets, semgrep for patterns). Fail build on critical findings. Prioritise: CVSS $\geq$ 9.0 = block, 7-8.9 = warn. Track false-positive rates and tune rules. Security gate must not add >2 min to pipeline for developer adoption.

Q48. How do you validate a vulnerability found in SAST (backend code) vs DAST?

SAST finding: trace data flow from source to sink in code. Confirm no sanitisation. Create a manual test case to reproduce in dev environment. DAST: automate with repeatable payload. Cross-reference: if SAST shows injection point and DAST confirms exploitability $\rightarrow$ true positive. Triage matrix: exploitability × business impact = priority.

Q49. Discovery tools – DAST/CI/CD/SCA.

DAST: ZAP (free), Burp Suite Enterprise, Qualys WAS, Invicti. CI/CD security: Trivy (containers), Checkov (IaC), tfsec, kube-bench. SCA: Snyk, OWASP Dependency-Check, Mend (WhiteSource). Secrets scanning: truffleHog, gitleaks, GitGuardian. Container: Grype, Clair, Anchore.

Q50. What is CrowdStrike (and endpoint security)?

CrowdStrike Falcon: cloud-native EDR/XDR platform. Lightweight agent collects telemetry, ML-based threat detection, threat hunting, incident response. Key features: real-time protection, threat intelligence, identity protection (Falcon Identity), device control. In interviews: distinguish EDR (Endpoint Detection & Response) from traditional AV—EDR is behavioural, not signature-only.

Q51. QRadar Architecture.

IBM QRadar SIEM. Components: Event Collector (collects log data), Event Processor (normalises, correlates), Flow Collector/Processor (network flows), Console (UI/query engine), Data Node (storage). Uses DSMs (Device Support Modules) for log parsing. Correlation rules → offenses. Custom rules using AQL (Ariel Query Language). Integrates with SOAR (QRadar SOAR / Resilient) for automated response.

Q52. Pattern Recognition in security.

Identifying anomalies in logs, network traffic, or code that indicate attacks. Techniques: baseline + deviation, ML anomaly detection, UEBA (User and Entity Behaviour Analytics), regex in SIEM rules, YARA rules for malware. Example: user logging in from two countries simultaneously (impossible travel) → alert.

---

6. Mobile Security (iOS / Android)

Q53. Android Components and their security implications.

Activity: UI screen (exported=true exposes to other apps). Service: background process (explicit intents preferred). BroadcastReceiver: listens for system/custom events (exported=true + no permissions = dangerous). ContentProvider: shares data across apps (SQL injection via URIs, unauthorised access if exported without permissions). Intent: inter-component messaging (intent redirection, fragment injection). Test: check AndroidManifest.xml for exported components.

Q54. How does mobile app redirection work (App to App – App Gray)?

Deep links / URI schemes: app registers a custom scheme (myapp://). Another app or browser can trigger it. Attack: open redirect via deep link, parameter tampering. Android App Links (https:// + Digital Asset Links verification) are harder to hijack than custom schemes. iOS Universal Links similar. App-to-App communication via Intents (Android) can leak data if exported Activities accept arbitrary intents.

Q55. How to exploit exported Activities (exported=true)?

Use adb: adb shell am start -n com.pkg/.ActivityName --es extra_value 'payload'. Or craft a malicious app that sends an Intent to the exported activity. Impact: bypass authentication screens, trigger privileged functionality, steal data passed via intent extras. Always check intent-filter and permission attributes in manifest.

Q56. Manual Jailbreak on iOS.

Jailbreaking removes Apple's sandbox restrictions using kernel exploits. Tools: checkra1n (hardware-based, semi-tethered), unc0ver, Palera1n. After jailbreak: install Frida, SSL Kill Switch 2, Objection for dynamic analysis. Security check: detect jailbreak in app via file system checks (/Applications/Cydia.app), dylib injection detection, sandbox integrity checks—but these are bypassable.

Q57. Interesting Vulnerabilities to discuss in interviews.

(1) Log4Shell (CVE-2021-44228): JNDI injection via log messages. (2) Spring4Shell. (3) Heartbleed: OpenSSL buffer over-read leaks server memory. (4) ShellShock: Bash env variable code execution. (5) POODLE/BEAST: TLS downgrade. (6) HTTP Request Smuggling in major CDNs. (7) OAuth misconfiguration leading to account takeover. (8) Dependency Confusion attacks. Discuss: root cause, impact, CVE, fix.

Q58. DOM XSS – Prevention and Functions to use/avoid.

Avoid: innerHTML, outerHTML, document.write, eval(), setTimeout(string), location.href=userInput, jQuery html(). Use instead: textContent, setAttribute (for non-URI attributes), createElement+appendChild, DOMPurify for sanitisation. Trusted Types API (browser enforcement). CSP with require-trusted-types-for 'script'.

---

7. Cloud & Container Security

Q59. Kubernetes / Docker Pentesting.

K8s attack surface: RBAC misconfiguration (default ServiceAccount), unauthenticated API server (port 6443/8080), etcd exposure, privileged containers, hostPath mounts, container escape (CVE-2019-5736 runc). Docker: privileged flag, docker.sock exposure (full host compromise), image vulnerabilities, secrets in ENV variables. Tools: kube-hunter, kube-bench, Trivy, Falco (runtime), Deepce.

Q60. Prototype Pollution in Node.js.

Via merge functions: obj.__proto__.isAdmin = true affects all objects. In Node.js can lead to RCE via gadget chains (e.g., child_process in template engines). Tools: NodeJSScan, semgrep rules. Fix: Object.freeze(Object.prototype), use safe merge libraries, JSON schema validation.

Q61. Exploiting SSL Certificates.

Attack vectors: use expired/self-signed certs → browser warning but MITM possible if user clicks through. Certificate mis-issuance (log4j in cert fields). BGP hijacking + fraudulent cert. Sub-domain takeover leading to cert issuance for that sub-domain. Wildcard cert compromise. Monitor via Certificate Transparency logs (crt.sh). Pin certificates in mobile/desktop apps.

Q62. Lateral Movement vs Network Pivoting.

Lateral Movement: moving from one compromised system to others in the same network (PtH, PtT, SMB relay, RDP). Pivoting: using a compromised host as a relay to reach otherwise inaccessible network segments (SSH tunnels, SOCKS proxy, Meterpreter route). Tools: Chisel, ligolo-ng, Metasploit route.

Q63. CVE vs CWE.

CVE (Common Vulnerabilities & Exposures): unique identifier for a specific vulnerability in a specific product (e.g., CVE-2021-44228 = Log4Shell). CWE (Common Weakness Enumeration): category of weakness types (e.g., CWE-79 = XSS, CWE-89 = SQLi). CVE instances are manifestations of CWEs. CVSS scores CVEs. OWASP categories map to CWEs.

Q64. OWASP categories for each vulnerability.

XSS → A03 Injection (CWE-79). SQLi → A03 Injection (CWE-89). IDOR → A01 Broken Access Control. Insecure Deserialisation → A08 Software and Data Integrity Failures. XXE → A03 Injection. SSRF → A10. Hardcoded credentials → A07 Auth Failures. Using known vulnerable library → A06. Security misconfiguration → A05.

---

8. Tools & Automation

Q65. Source Code Review – methodology.

(1) Understand the tech stack. (2) Map entry points (params, headers, file uploads, APIs). (3) Trace data flow from sources to sinks. (4) Check for: missing validation, hardcoded secrets, insecure crypto, dangerous functions. (5) Use SAST tools (Semgrep, CodeQL) + manual review for logic flaws. (6) Focus on authentication, authorisation, cryptography, serialisation code. Triages: severity, exploitability, business impact.

Q66. Sqlmap features.

Automated SQLi detection and exploitation: --dbs (list databases), --tables, --dump, --os-shell (OS command execution if stacked queries), --file-read, --batch (non-interactive), -p (specific parameter), --level/--risk (thoroughness), --tamper (WAF bypass scripts), --cookie, --headers. Always use with permission. Post-exploitation: extract password hashes, pivot via UDF.

Q67. Security Automation with LLM integration.

LLMs (GPT-4, Claude) in security pipelines: triage SAST false positives, generate remediation suggestions, automate threat modelling (STRIDE prompts), generate test cases from code diff, natural language search in SIEM. Concerns: hallucinations in security context → always human-validate LLM output. Prompt injection risks in AI-assisted code review tools.

Q68. Automation in Security – general.

CI/CD security gates (SAST/SCA/secrets). Automated DAST on staging. SOAR playbooks for alert triage (enrich IOC → block IP → notify). Custom scripts for recon (subfinder + httpx + nuclei). Threat intel enrichment (VirusTotal API, Shodan). Bug bounty automation: subfinder, amass, nuclei templates.

Q69. SCA tools – open source library scanning in company.

Integrate Snyk or OWASP Dependency-Check in build pipeline. Generate SBOM (CycloneDX or SPDX format). Alert on CVSS ≥ 7. Track licence compliance (GPL conflicts). Dependency-Track for SBOM management. Auto-PRs for upgrades (Dependabot, Renovate). Triage: is vulnerable function actually called? (reachability analysis in Snyk).

Q70. Programming for Security Engineers.

Python: scripting, Burp extensions (Jython), exploit PoCs, automation. Bash: recon pipelines, CI/CD glue. JavaScript: XSS payloads, Node.js auditing. Java/C#: reviewing enterprise code. SQL: understanding injection, writing detection queries. Regex: SAST rules, log parsing. Know: requests library, BeautifulSoup, Scapy, pwntools, impacket.

---

9. Miscellaneous Technical Topics

Q71. Horizontal vs Vertical Privilege Escalation.

Horizontal: access another user's resources at the same privilege level (IDOR – change userId=123 to userId=124). Vertical: escalate to higher privileges (normal user → admin). Test: map all roles, try accessing higher-privilege endpoints with lower-privilege tokens. Remediation: server-side authorisation checks on every request, not just at login.

Q72. Deserialization vulnerabilities.

Insecure deserialisation: untrusted data is deserialised into objects → RCE or object injection. Java: ysoserial gadget chains (Commons Collections). PHP: POP chains via __wakeup/__destruct. Python: pickle.loads. Node.js: node-serialize. Detection: look for ObjectInputStream, pickle, unserialize in code. Mitigation: avoid deserialising untrusted data, use JSON instead of binary serialisation, sign serialised data, whitelist classes.

Q73. Thin client security.

Thin clients rely on server-side processing. Attack surface: unencrypted RDP/VNC sessions, session hijacking, clipboard data leakage, USB redirection exploitation, default credentials on management interface. Harden: enforce TLS, MFA, disable unused I/O redirection, network segmentation.

Q74. RACE condition in bug bounty context.

Send parallel requests to the same endpoint (e.g., apply coupon code 50 times simultaneously). Turbo Intruder (Burp) for low-latency parallel requests. Single-packet attack (HTTP/2 multiplexing) for true simultaneous requests. Impact: coupon reuse, double withdrawal, duplicate reward points. Mitigation: DB-level locks, atomic test-and-set, idempotency keys.

Q75. Interesting findings: exploiting certificate transparency and sub-domain takeover.

Monitor crt.sh / Cert Spotter for new certs issued for your domain. Enumerate sub-domains → check for dangling CNAMEs pointing to deprovisioned cloud services (S3, GitHub Pages, Heroku). Claim the deprovisioned resource → issue a cert → serve phishing page or steal cookies (if cookie domain is overly broad). Tools: subzy, subjack, nuclei sub-takeover templates.

Q76. HTTP Only – bypassing.

HttpOnly prevents JS access to cookie. Bypass: find XSS that causes server-side response including the cookie value, exploit CSRF instead of XSS to perform actions without reading cookie, TRACE method (Cross-Site Tracing – disabled in modern browsers), find a browser bug. Best note: HttpOnly reduces risk but is not absolute.

Q77. Port 53 – test cases.

DNS pentesting: zone transfer (dig axfr @ns.target.com target.com), DNS cache poisoning test, DNS amplification (open resolver check), DNSSEC validation, subdomain enumeration (dnsx, massdns), DNS tunnelling (iodine), reverse lookup bruteforce, zone walking (NSEC). Check for split-horizon DNS that exposes internal names.

Q78. Vulnerabilities in POS System in Supermarket.

Network: POS on flat network with payment processor → breach risk. Attacks: memory-scraping malware (RAM parsing for Track 1/2 card data between swipe and encryption), weak point-to-point encryption, default credentials, unpatched Windows XP/Embedded. Also: physical tampering (skimmer), insecure Wi-Fi. PCI DSS requirements: P2PE, network segmentation, patch management, logging.

Q79. Send multiple payloads without sending multiple requests.

Techniques: HTTP pipeline (multiple requests in one TCP connection), HTTP/2 multiplexing (multiple streams in one connection), batch API endpoints, XML/JSON with array of payloads, WebSocket for persistent channel. In SQLi: stacked queries if supported. SSRF: send multiple URLs via CRLF-delimited gopher:// payloads.

Q80. Cookies (null, strict, lax).

SameSite=None: cookie sent on all cross-site requests (requires Secure flag). SameSite=Strict: never sent cross-site. SameSite=Lax: sent on top-level navigation (e.g., clicking a link) but not in sub-resource requests (images, iframes). Default (no SameSite): browser-dependent; modern browsers default to Lax. Null/missing SameSite in legacy browsers = sent on all cross-origin requests.

Q81. Automations – Security (Security JMX / Python).

JMeter: load + basic security testing (fuzz inputs). Python requests: custom scanner scripts. Nuclei: template-based mass scanning. Playwright/Selenium: authenticated automated DAST. GitHub Actions + OWASP ZAP: nightly scan of staging. Custom Burp extensions in Python (Jython) for repetitive tasks. Metrics: mean time to detect (MTTD), false-positive rate.

Q82. Security Headers – HTTP.

Must-have: Strict-Transport-Security (max-age=31536000; includeSubDomains; preload), Content-Security-Policy, X-Frame-Options: DENY (or CSP frame-ancestors), X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy. Check via securityheaders.com. In APIs: remove Server, X-Powered-By (information disclosure).

Q83. Different HTTP Methods.

GET: retrieve (idempotent, no body). POST: create/submit. PUT: replace. PATCH: partial update. DELETE: delete. OPTIONS: CORS preflight, discover allowed methods. HEAD: GET without body. TRACE: echo (XST – should be disabled). CONNECT: HTTP tunnelling. Security: test all methods on each endpoint. Verify IDOR not exploitable via method override (X-HTTP-Method-Override).

PART B — BEHAVIORAL / HR / SITUATIONAL QUESTIONS

QB1. How do you explain technical concepts to non-technical people?

Use analogies: 'XSS is like leaving a malicious note on a bulletin board that others unknowingly execute.' Structure: problem → real-world impact → plain-language solution. Avoid jargon. Use visuals when possible. Check comprehension by asking follow-up questions. Tailor depth to audience role (exec vs developer).

QB2. What are possible attack surfaces in a given architecture?

Walk through: external-facing endpoints, authentication mechanisms, third-party integrations, data stores, inter-service communication, CI/CD pipeline, DNS/PKI, cloud configuration, employee endpoints. Use threat modelling frameworks: STRIDE, PASTA, attack trees. Demonstrating structured thinking matters more than listing every attack.

QB3. How do you resolve conflict in your team?

Active listening first. Separate people from the problem. Focus on shared goal (secure product). Data-driven: back positions with evidence (CVE data, threat model). Escalate to manager only after peer resolution fails. Example: developer pushes back on SAST findings—schedule a quick call to walkthrough exploitability together.

QB4. Name a time you failed at something and how you overcame it.

Structure with STAR (Situation, Task, Action, Result). Choose a genuine example. Emphasise the learning. Show it changed your process (e.g., missed a vulnerability in review → added automated SAST rule, ran retro with team). Demonstrate growth mindset.

QB5. Tell me about yourself.

Education + technical background (highlight relevant certifications: OSCP, CEH, GWEB, AWS Security). Work experience (2-3 relevant highlights with impact metrics). Personal interest in security (CTFs, bug bounty, research). Big picture: connect your goals to the company's mission/product. Keep it to 2-3 minutes.

QB6. Why this job / role / company?

Research the company: product, security posture, recent breaches or security initiatives, team blog posts. Connect to your strengths. Example: 'Your platform handles payments for X million users—the AppSec challenges in that scale of PCI-DSS environment directly match my experience with...'. Show genuine interest, not generic answers.

QB7. What are your strengths as a security engineer?

Pick 2-3 strengths with concrete evidence: e.g., 'Systematic thinker – I build threat models before touching a keyboard.' 'Developer empathy – I write remediation PoCs so devs understand the fix, not just the vulnerability.' 'Automation-first – I reduced manual pen-test effort by 40% with a custom Nuclei template library.'

QB8. What would you improve to be a better pentester?

Show self-awareness and proactive learning: 'I want to deepen my binary exploitation skills (currently doing pwn.college). Improve cloud-native attack techniques—pursuing AWS Security Specialty. Contribute more to OSS tools to give back to the community.' Show a concrete plan.

QB9. What are you looking for in a future company (day-to-day tasks)?

Be honest and specific: 'I want hands-on technical work (not just report writing). Exposure to threat modelling at design phase. Collaboration with engineering (shift-left). A team that invests in tooling and automation. Clear escalation paths and management support for security decisions.'

QB10. If a vulnerability is in a 3rd-party component with no official patch, what do you do?

(1) Assess exploitability in your specific usage context. (2) Apply virtual patching (WAF rule, network control). (3) Isolate the component (network segmentation). (4) Monitor for exploit activity. (5) Notify vendor, track their advisory. (6) Evaluate alternative libraries. (7) Document risk acceptance with business owner if workaround is temporary.

QB11. You find a RCE on a public-facing site, but developers hesitate to take it down—what do you do?

(1) Clearly communicate business impact (data breach risk, regulatory penalties). (2) Quantify: what data is accessible, what's the blast radius. (3) Offer minimum viable mitigation (WAF rule, feature flag to disable the endpoint). (4) Escalate to CISO/CTO with written risk statement. (5) Document your recommendation and their decision (CYA). Never exploit beyond PoC.

QB12. OWASP categories and automation + LLM integration.

Modern AppSec pipelines map findings to OWASP categories automatically (Semgrep rules tag CWE→OWASP). LLM can suggest which OWASP category a finding belongs to from the code snippet. Automate OWASP Top 10 coverage reporting per sprint. Use OWASP Benchmark to measure tool accuracy.

QB13. Recent project – challenges and outcome.

Use STAR. Metrics: 'Reduced critical vulns in CI/CD from 47 to 3 in one quarter by implementing SCA gate and developer training.' Challenges: developer buy-in, false-positive fatigue. Outcome: improved MTTD, security KPIs added to quarterly OKRs.

QB14. Personal projects.

CTF writeups, bug bounty reports (HackerOne/Bugcrowd), custom tooling on GitHub, security blog, CVE submissions, Hack The Box / TryHackMe labs, contributing to Nuclei templates or OWASP projects. Shows passion beyond 9-5.

QB15. What is your role in LLM implementation / AI security?

LLM security concerns: prompt injection, training data poisoning, model inversion, insecure plugin integration, OWASP LLM Top 10. Your role: threat model the LLM pipeline, red-team prompts, enforce output validation, restrict tool/function calling permissions, monitor for anomalous completions, apply DLP to prompts containing PII.

PART C — QUESTIONS TO ASK YOUR INTERVIEWER

1. How big is the security team, and what are the sub-team structures?

2. Is this a new role or replacing someone—what's the history of this position?

3. What does a typical week look like for someone in this role?

4. How mature is the SDLC security integration today, and what's the roadmap?

5. What is the biggest security challenge the team is currently facing?

6. How do you measure success for this role in the first 90 days / 6 months?

7. What's the relationship between security and engineering teams?

8. What tools and budget does the team have access to?

9. How does the company handle vulnerability disclosure from external researchers?

10. Is there a security champions programme or developer security training?

PART D — BONUS QUESTIONS [Claude's Additions]

The following questions are added by Claude based on common AppSec/ProdSec/Security Engineer interview patterns. They are NOT from the original notes.

Bonus – Advanced Web Security

QD1. [Claude Addition] Explain HTTP/2 vs HTTP/1.1 from a security perspective.

HTTP/2: multiplexing (multiple streams per TCP), header compression (HPACK), binary framing. Security delta: HTTP/2 enables single-packet attack for race conditions (no network jitter). HPACK compression side-channel (CRIME-like). Request smuggling: H2.CL, H2.TE variants. Browsers require TLS for HTTP/2 (h2) in practice. HTTP/2 cleartext (h2c) for internal services—often misconfigured.

QD2. [Claude Addition] What is IDOR and how do you find and fix it?

IDOR (Insecure Direct Object Reference): A01 Broken Access Control. Attacker changes a reference (userId, orderId) to access another user's resource. Find: map all object references, test horizontal and vertical with two accounts (Autorize Burp extension). Fix: server-side authorisation on every data access, indirect references (UUID instead of sequential int, but this is obscurity—not a fix by itself).

QD3. [Claude Addition] Explain Business Logic vulnerabilities.

Flaws in the application's intended workflow. Examples: negative price in shopping cart, skip payment step, coupon stacking, referral self-abuse, password reset flaw (predictable token). Cannot be found by scanners—require manual testing and understanding of business rules. Document expected flow → test every deviation.

QD4. [Claude Addition] What is GraphQL and what are its security issues?

GraphQL: query language for APIs, single endpoint. Issues: introspection leaks full schema (disable in production), excessive data exposure (ask for all fields), query depth/complexity DoS, IDOR via nested queries, batch query abuse, injections via variables. Tools: InQL (Burp), clairvoyance (schema reconstruction without introspection).

QD5. [Claude Addition] Explain OWASP LLM Top 10.

LLM01 Prompt Injection, LLM02 Insecure Output Handling, LLM03 Training Data Poisoning, LLM04 Model Denial of Service, LLM05 Supply Chain Vulnerabilities, LLM06 Sensitive Information Disclosure, LLM07 Insecure Plugin Design, LLM08 Excessive Agency, LLM09 Overreliance, LLM10 Model Theft. Key interview point: distinguish direct (user→LLM) vs indirect prompt injection (attacker embeds in retrieved document).

Bonus – Cloud & DevSecOps

QD6. [Claude Addition] What is the Shared Responsibility Model in cloud security?

AWS/Azure/GCP: provider secures 'of' the cloud (physical, hypervisor, managed services). Customer secures 'in' the cloud (OS, application, data, IAM, network config). Misunderstanding this model is the root cause of many cloud breaches (misconfigured S3, over-permissive IAM, unpatched EC2).

QD7. [Claude Addition] What are the top cloud security misconfigurations you look for?

S3/GCS/Blob storage: public read/write. IAM: wildcard permissions, no MFA on root/privileged accounts, over-permissive service accounts. Security groups: 0.0.0.0/0 on SSH/RDP. KMS: key rotation disabled. CloudTrail/Audit Logs: disabled. Lambda: environment variables with secrets. ECR images: public, unscanned. Metadata service: IMDSv1 (SSRF to steal credentials).

QD8. [Claude Addition] Explain Infrastructure as Code (IaC) security.

Terraform, CloudFormation, Helm charts: codified infra—security issues version-controlled and replicated at scale. Scan with: Checkov, tfsec, Terrascan, KICS. Common issues: open security groups, unencrypted storage, missing audit logging, hardcoded secrets. Shift-left: scan IaC in PR before apply. Enforce via OPA/Sentinel policies.

Bonus – Incident Response & Threat Modelling

QD9. [Claude Addition] How do you perform a Threat Model?

Frameworks: STRIDE (Spoofing, Tampering, Repudiation, Info Disclosure, DoS, Elevation of Privilege) or PASTA (Process for Attack Simulation and Threat Analysis). Steps: (1) Define scope and assets. (2) Create DFD (Data Flow Diagram). (3) Identify trust boundaries. (4) Enumerate threats per component. (5) Rate by likelihood × impact. (6) Define mitigations. Tooling: OWASP Threat Dragon, IriusRisk, Miro.

QD10. [Claude Addition] Walk me through your incident response process.

NIST IR phases: Preparation → Detection & Analysis → Containment → Eradication → Recovery → Post-Incident. Key: preserve evidence (forensic image before cleanup), timeline reconstruction, IOC extraction, communication plan (legal, PR, regulatory notification within 72h under GDPR). Post-mortem: blameless, root cause analysis, control improvements.

QD11. [Claude Addition] What is Supply Chain Attack? Give an example.

Attacker compromises a trusted third-party component to reach the ultimate target. Examples: SolarWinds (malicious update), XZ Utils backdoor (CVE-2024-3094), npm package hijacking (event-stream), dependency confusion. Mitigations: pin exact dependency versions, verify checksums/signatures, SBOM, private package registry, review transitive dependencies.

Bonus – Certifications & Career

QD12. [Claude Addition] Which security certifications are most valued for AppSec/ProdSec roles?

Offensive: OSCP (OffSec), eWPT/eWPTX (web AppSec), BSCP (Burp Suite Certified Practitioner). Defensive/AppSec: GWEB (GIAC Web Application Penetration Tester), CSSLP (Certified Secure Software Lifecycle Professional). Cloud: AWS Security Specialty, GCP Professional Cloud Security Engineer. General: CEH, CompTIA Security+, CISSP (senior roles). Bug bounty hall-of-fame is equally respected.

QD13. [Claude Addition] How do you stay current in cybersecurity?

Follow: PortSwigger Blog, Project Zero, NVD/CVE feeds, OWASP, HackerOne disclosed reports, Hacking Articles. Read: The Daily Swig, DarkReading, KrebsOnSecurity. Practice: HackTheBox, TryHackMe, DVWA, Juice Shop, pwn.college. Conferences: DEF CON, Black Hat, OWASP AppSec. Twitter/X: @taviso, @LiveOverflow, @infosec_au. CTF competitions.